This tutorial will create a Flask CRUD application that allows users to create, read, update, and delete database entries using an API. The API will be able to:

- **List all instances of object**
- **Post a new instance**
- **Get a specific instance**
- **Put a specific instance**
- **Delete a specific instance**

Tutorial Steps

1. Project Setup:
   - Create PostgreSQL Database
   - Initialize the Virtual Environment
   - Install the Project Dependencies
2. Writing the Project Code:
   - Writing the Main Files
   - Writing the Applications Files
   - Send Requests Using Postman

Definitions

> 💡 *Tip: Skip these definitions at the first reading time!*

- **What is Flask?**

> Flask is what is known as a **WSGI framework**. Which stands for **Web Server Gateway Interface**. Essentially, this is a way for web servers to pass requests to web applications or frameworks.

Flask is used for developing web applications using Python. Advantages of using Flask framework:
- Lightweight framework.
- Use **MVC** design pattern.
- Has a built-in development server.
- Fast debugger is provided.

- **What is SQLAlchemy?**

SQLAlchemy provides a nice "**Pythonic**" way of interacting with databases.

SQLAlchemy is a library that facilitates the communication between Python programs and databases. Most of the time this library is used as an **Object Relational Mapper** (**ORM**) tool that ***translates Python classes to tables*** in relational databases and automatically ***converts function calls to SQL statements***.

- **What is Alembic?**

Alembic is a lightweight database migration tool for usage with the SQLAlchemy Database Toolkit for Python.

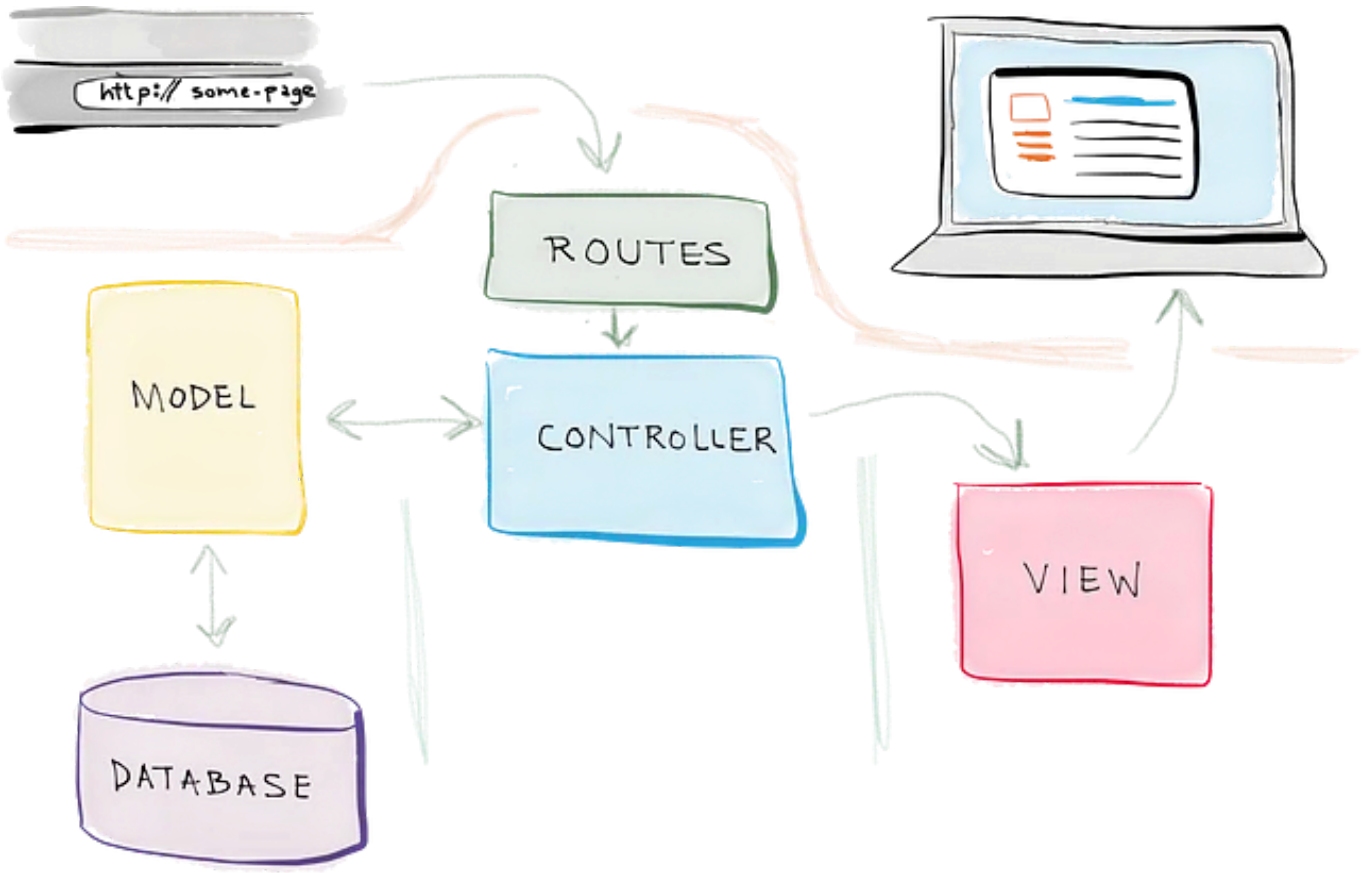
Alembic is a very useful library which is widely used for **database migration**. It can be used to create tables, insert data or even migrate functions from one schema to another. To be able to do all these tasks, the library uses SQLAlchemy, an ORM that is suited for working with PostgreSQL and other relational databases.

- **MVC Design Pattern**

The Model-View-Controller (MVC) is an architectural pattern that separates an application into three main groups of components: **Models**, **Views**,

and **Controllers**.

MVC (Model-View-Controller) is a pattern in software design commonly used to implement user interfaces, data, and controlling logic. It emphasizes the separation between the software's business logic and display. This "separation of concerns" provides for a better division of labor and improved maintenance.



MVC

. . .

Prerequisites

- Windows Terminal
- Text Editor like VSCode
- Postman

- Python Interpreter
- PostgreSQL Server
- pgAdmin *“optional”*

. . .

Project Setup

#1 Create PostgreSQL Database

Target: Create a new database with a new user.

> 💡 *Tip: First create a test database with the same names & passwords below, then you can create a real database with the names & passwords you want!*

We will create a database called “**testdb**” and user “**testuser**” with password “**testpass**”.

1. In Windows Terminal, Run the PostgreSQL Server

```
~ sudo service postgresql start
➜ * Starting PostgreSQL 14 database server
# 14 is the PostgreSQL Server Version
```

> 📝 *Important Note: We need to run the PostgreSQL server every time we start coding!*

2. Activate the PostgreSQL Shell

```
~ sudo -u postgres psql
➜ postgres=#
```

3. Create a New Database

```
<!-- create database DBNAME; -->
postgres=# create database testdb;
➜ CREATE DATABASE
```

4. Create a Database User, then Grant Privileges to it

```
<!-- create user USERNAME with encrypted password 'PASSWORD'; -->
postgres=# create user testuser with encrypted password 'testpass';
➜ CREATE ROLE

<!-- grant all privileges on database DBNAME to USERNAME; -->
postgres=# grant all privileges on database testdb to testuser;
➜ GRANT
```

5. Exit the Shell

```
postgres=# \q
```

6. Connect to the New Database

```
~ psql -U testuser -h 127.0.0.1 -d testdb
Password for user testuser: testpass
```

```
➜ testdb=>
```

7. Check the Connection

```
testdb=> \conninfo
➜ You are connected to database "testdb" as user "testuser" on host "127.0.0.1"
<!-- We need this information later for the env file -->
```

Now that our new PostgreSQL database is up and running, let's move on to the next step!

#2 Initialize the Virtual Environment

- What is the Virtual Environment?

> A virtual environment is a tool that helps separate dependencies required by different projects by creating isolated python virtual environments for them. This is one of the most important tools that most Python developers use.
>
> virtualenv is used to manage Python packages for different projects. Using virtualenv allows you to avoid installing Python packages globally which could break system tools or other projects.

We'll create a virtual environment and activate it using the following commands

```
# virtualenv -p python3 ProjectName
~ virtualenv -p python3 Flask-SQLAlchemy-PostgreSQL
➜ created virtual environment

cd Flask-SQLAlchemy-PostgreSQL
```

```
source bin/activate
```

#3 Install the Project Dependencies

After creating and activating the virtualenv, let's start with installing the project's dependencies

```
pip install python-dotenv flask flask-sqlalchemy Flask-Migrate flask_validator p
```

Then make a folder called src which will contain the project codes

```
mkdir src && cd $_
```

The Last step before starting with the code, create a requirements file using this command:

```
python -m pip freeze > requirements.txt
```

. . .

Writing the Project Code

> 📝 *Note: In Flask, you can structure and name the files however you like, but we will learn the best practices for the naming and files structuring.*

```
├── bin
├── include
├── lib
├── pyvenv.cfg
└── src
    ├── config.py
    ├── .env
    ├── .env.sample
    ├── __init__.py
    ├── app.py
    ├── accounts
    │   ├── controllers.py
    │   ├── models.py
    │   └── urls.py
    ├── items
    │   ├── controllers.py
    │   ├── models.py
    │   └── urls.py
    ├── requirements.txt
    └── README.md
```

#1 Getting Started with the Main Files "`app`, `__init__`, `config`, `env`"

In most Flask tutorials, you'll notice that they only have the `app.py` file, which works. However, it is better to have multiple files, which makes the code clean and file management much easier, especially in large projects.

Get Yahia Qous's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

So, let's create the 4 main files with this command:

```
touch app.py __init__.py config.py .env
```

Now let's start diving deeper into each file:

> *Unpopular opinion: Better to start with* `config.py` *than* `app.py`

- ***`config.py`***

  Let's assume that we have 4 configuration modes: Development, Testing, Staging, and Production. We will create a class for each one with the configuration values, you can check the Configuration — Flask-SQLAlchemy Documentation. The most important one is `SQLALCHEMY_DATABASE_URI` which is equal to the PostgreSQL database connection link.

```
import os
class Config:
    SQLALCHEMY_TRACK_MODIFICATIONS = True
class DevelopmentConfig(Config):
    DEVELOPMENT = True
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.getenv("DEVELOPMENT_DATABASE_URL")
class TestingConfig(Config):
    TESTING = True
    SQLALCHEMY_DATABASE_URI = os.getenv("TEST_DATABASE_URL")
class StagingConfig(Config):
    DEVELOPMENT = True
    DEBUG = True
    SQLALCHEMY_DATABASE_URI = os.getenv("STAGING_DATABASE_URL")
class ProductionConfig(Config):
    DEBUG = False
    SQLALCHEMY_DATABASE_URI = os.getenv("PRODUCTION_DATABASE_URL")
config = {
    "development": DevelopmentConfig,
    "testing": TestingConfig,
    "staging": StagingConfig,
    "production": ProductionConfig
}
```

- ***`.env`***

  Create the environment variables for the config mode and the database

URL for each mode.

```
# Configuration Mode => development, testing, staging, or production
CONFIG_MODE = development

# POSTGRESQL_DATABASE_URI => 'postgresql+psycopg2://user:password@host:port/data
DEVELOPMENT_DATABASE_URL = 'postgresql+psycopg2://testuser:testpass@localhost:54
TEST_DATABASE_URL        =
STAGING_DATABASE_URL     =
PRODUCTION_DATABASE_URL  =
```

PostgreSQL database connection URL format `postgresql+psycopg2://user:password@host:port/database`. This information can be obtained using `\conninfo` command in the psql shell.

- ***`__init__.py`***

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

from .config import config

db = SQLAlchemy()
migrate = Migrate()

def create_app(config_mode):
    app = Flask(__name__)
    app.config.from_object(config[config_mode])

    db.init_app(app)
    migrate.init_app(app, db)

    return app
```

`create_app` is a function that instantiates:

1. app from the Flask class with the configs from the `config.py` file we created.
2. db from SQLAlchemy class imported from flask_sqlalchemy.
3. migrate from Migrate class imported from flask_migrate.

- ***app.py***

```
import os

# App Initialization
from . import create_app # from __init__ file
app = create_app(os.getenv("CONFIG_MODE"))

# Hello World!
@app.route('/')
def hello():
    return "Hello World!"

if __name__ == "__main__":
    app.run()
```

Now our basic app is ready to go! We can run the server in the terminal by using one of the following commands:

```
# To Run the Server in Terminal
flask run

# To Run the Server with specific host and port
# flask run -h HOSTNAME -p PORTNUMBER
flask run -h 127.0.0.2 -p 5001

# To Run the Server with Automatic Restart When Changes Occur
FLASK_DEBUG=1 flask run
```

You can open your browser at http://127.0.0.1:5000 and see the result!

#2 Getting Started with the Applications Files

All the pains and headaches above are for the first time starting the project; most code is written inside the files of the applications.

> 💡 *Tip: It is a best practice to have each app in a separate folder.*

Each app should have its own **models**, **urls**, and **controllers**.

Let's start by creating an app called Accounts with this command:

```
mkdir accounts && touch $_/models.py $_/urls.py $_/controllers.py
```

Now, let's break down all these files:

> 💡 *Tip: Always start with building the models classes*

- `models.py`

```
from sqlalchemy import inspect
from datetime import datetime
from flask_validator import ValidateEmail, ValidateString, ValidateCountry
from sqlalchemy.orm import validates

from .. import db # from __init__.py

# ------------------------------------------------ #

# SQL Datatype Objects => https://docs.sqlalchemy.org/en/14/core/types.html
class Account(db.Model):
# Auto Generated Fields:
    id          = db.Column(db.String(50), primary_key=True, nullable=False, un
    created     = db.Column(db.DateTime(timezone=True), default=datetime.now)
    updated     = db.Column(db.DateTime(timezone=True), default=datetime.now, c

# Input by User Fields:
```

```
    email        = db.Column(db.String(100), nullable=False, unique=True)
    username     = db.Column(db.String(100), nullable=False)
    dob          = db.Column(db.Date)
    country      = db.Column(db.String(100))
    phone_number = db.Column(db.String(20))


# Validations => https://flask-validator.readthedocs.io/en/latest/index.html
    @classmethod
    def __declare_last__(cls):
        ValidateEmail(Account.email, True, True, "The email is not valid. Please
        ValidateString(Account.username, True, True, "The username type must be
        ValidateCountry(Account.country, True, True, "The country is not valid")


# Set an empty string to null for username field => https://stackoverflow.com/a/
    @validates('username')
    def empty_string_to_null(self, key, value):
        if isinstance(value, str) and value == '': return None
        else: return value


# How to serialize SqlAlchemy PostgreSQL Query to JSON => https://stackoverflow.
    def toDict(self):
        return { c.key: getattr(self, c.key) for c in inspect(self).mapper.colum

    def __repr__(self):
        return "<%r>" % self.email
```

- **`controllers.py`**

  The general CRUD requests are:
  - List all instances
  - Post a new instance
  - Get a specific instance
  - Put a specific instance
  - Delete a specific instance

Each of these operations must have its own logical function in the `controllers.py` file:

```python
from flask import request, jsonify
import uuid

from .. import db
from .models import Account

# ---------------------------------------------- #

# Query Object Methods => https://docs.sqlalchemy.org/en/14/orm/query.html#sqlal
# Session Object Methods => https://docs.sqlalchemy.org/en/14/orm/session_api.ht
# How to serialize SqlAlchemy PostgreSQL Query to JSON => https://stackoverflow.

def list_all_accounts_controller():
    accounts = Account.query.all()
    response = []
    for account in accounts: response.append(account.toDict())
    return jsonify(response)

def create_account_controller():
    request_form = request.form.to_dict()

    id = str(uuid.uuid4())
    new_account = Account(
                          id             = id,
                          email          = request_form['email'],
                          username       = request_form['username'],
                          dob            = request_form['dob'],
                          country        = request_form['country'],
                          phone_number   = request_form['phone_number'],
                          )
    db.session.add(new_account)
    db.session.commit()

    response = Account.query.get(id).toDict()
    return jsonify(response)

def retrieve_account_controller(account_id):
    response = Account.query.get(account_id).toDict()
    return jsonify(response)

def update_account_controller(account_id):
    request_form = request.form.to_dict()
    account = Account.query.get(account_id)

    account.email        = request_form['email']
    account.username     = request_form['username']
    account.dob          = request_form['dob']
    account.country      = request_form['country']
    account.phone_number = request_form['phone_number']
    db.session.commit()

    response = Account.query.get(account_id).toDict()
```

```
    return jsonify(response)

def delete_account_controller(account_id):
    Account.query.filter_by(id=account_id).delete()
    db.session.commit()

    return ('Account with Id "{}" deleted successfully!').format(account_id)
```

Let's break down the logical functions for CRUD operations:

- List all instances:
  1. Get all queries using **query.all()** method
  2. Loop through the result to save the instances in a list of dictionaries
  3. Jsonify the list
- Post new instance:
  1. Get the request data sent in the request form and convert it into dictionary
  2. Create a unique id from uuid library
  => https://docs.python.org/3/library/uuid.html
  3. Create a new instance of the class with the request form data
  4. Add then Commit the session to save the new instance in our database
  5. Retrieve the new instance by **id** using **query.get()** method
  6. Convert the result into dictionary then Jsonify it
- Get a specific instance:
  1. Retrieve the instance by the **provided id** using **query.get()** method
  2. Convert the result into dictionary then Jsonify it
- Put a specific instance:
  1. Get the request data sent in the request form and convert it into dictionary
  2. Retrieve the instance by the **provided id** using **query.get()** method
  3. Update the instance fields with the request form data
  4. Commit the session to save the instance with the new data in our

database

5. Retrieve the instance by the **provided id** using **query.get()** method
6. Convert the result into dictionary then Jsonify it

- Delete a specific instance:

  1. Retrieve the instance by the **provided id** using **query.filter_by()** method
  2. Commit the session to take action in our database
  3. Return with a message to notify the user with the result

- `urls.py`

  The five general operations can be combined into two URLs like this:

```
from flask import request

from ..app import app
from .controllers import list_all_accounts_controller, create_account_controller

@app.route("/accounts", methods=['GET', 'POST'])
def list_create_accounts():
    if request.method == 'GET': return list_all_accounts_controller()
    if request.method == 'POST': return create_account_controller()
    else: return 'Method is Not Allowed'

@app.route("/accounts/<account_id>", methods=['GET', 'PUT', 'DELETE'])
def retrieve_update_destroy_accounts(account_id):
    if request.method == 'GET': return retrieve_account_controller(account_id)
    if request.method == 'PUT': return update_account_controller(account_id)
    if request.method == 'DELETE': return delete_account_controller(account_id)
    else: return 'Method is Not Allowed'
```

Now, two steps are required to get our accounts app ready to go:

1. Import the `urls` file in the `app.py`

   The final shape of the `app.py` file should look like this:

```
import os

# App Initialization
from . import create_app # from __init__ file
app = create_app(os.getenv("CONFIG_MODE"))

# ------------------------------------------------ #

# Hello World!
@app.route('/')
def hello():
    return "Hello World!"

# Applications Routes
from .accounts import urls

# ------------------------------------------------ #

if __name__ == "__main__":
    # To Run the Server in Terminal => flask run -h localhost -p 5000
    # To Run the Server with Automatic Restart When Changes Occurred => FLASK_DE

    app.run()
```

2. Migrate the new database models with these commands:

```
flask db init
flask db migrate
flask db upgrade
```

If you face this error: AttributeError: `'_FakeStack'` object has no attribute `'__ident_func__'`, then fix it with these commands:

```
python -m pip uninstall flask-sqlalchemy
python -m pip install flask-sqlalchemy
```

You can learn more about the Flask-Migrate library from https://flask-migrate.readthedocs.io/en/latest

#3 Send Requests Using Postman

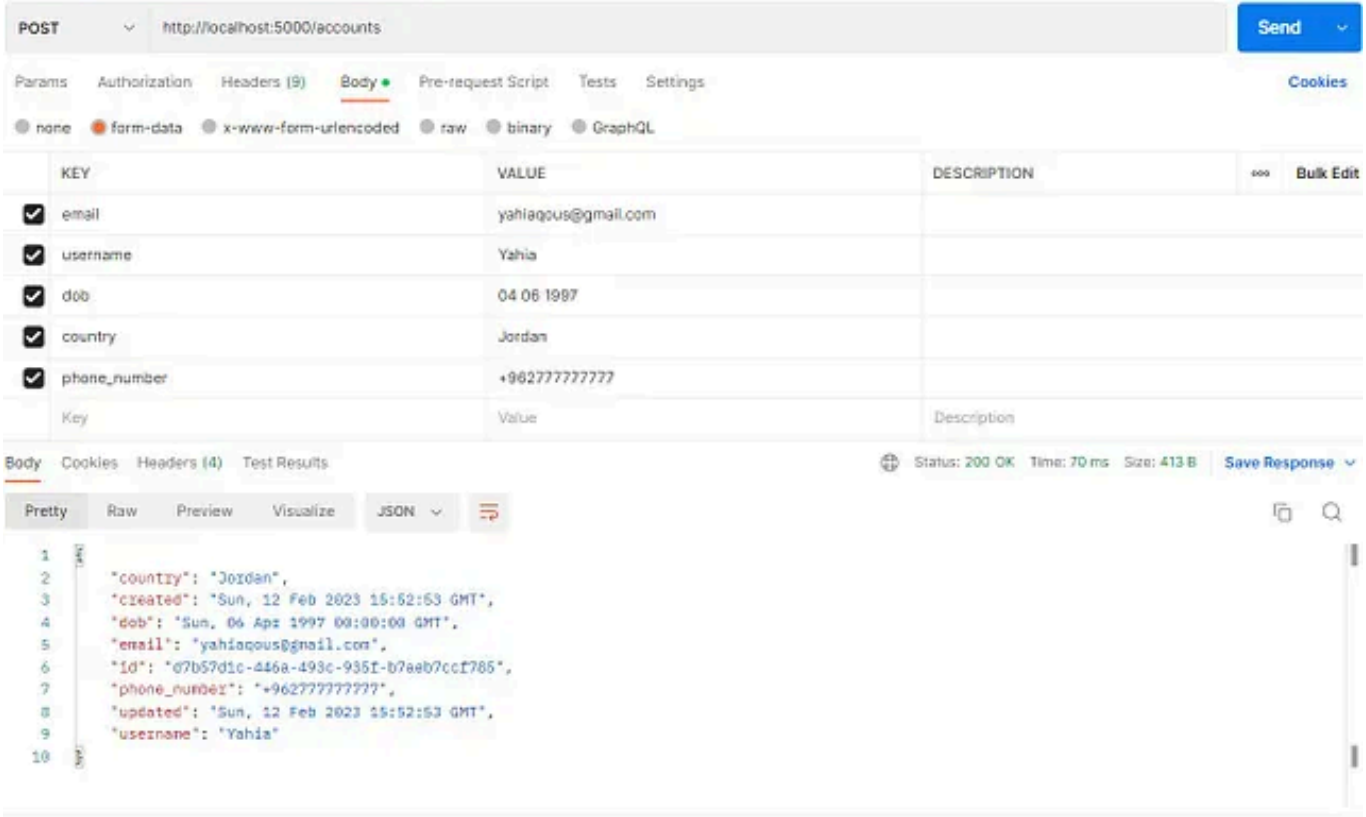
In this section, we will use Postman to test all of the CRUD operations we created.

What is Postman?

> *Postman is an application that allows us to do API testing. It's like a browser that doesn't render HTML. In the browser, we can hit only GET HTTP requests but here we can hit GET, POST, PUT, DELETE, and many more HTTP requests in API.*
>
> *Postman is the world's largest public API hub. It's an API platform for developers to design, build, test, and iterate their own APIs.*

1. **Post New Account:**

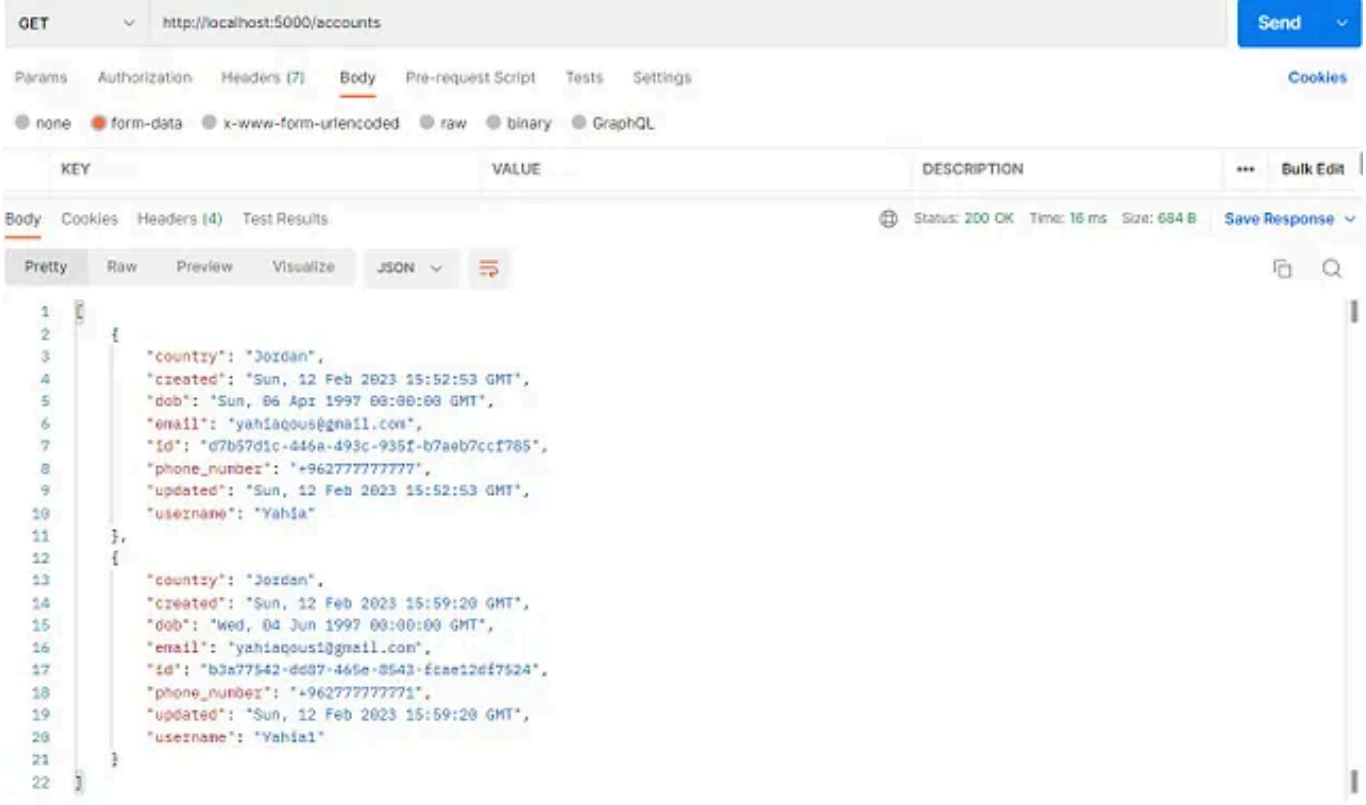
- Request Method: **POST**
- Request Link: **http://localhost:5000/accounts**
- Body Data in form-data:
  - email
  - username
  - dob
  - country
  - phone_number



Postman - Post New Account

2. **List All Accounts:**

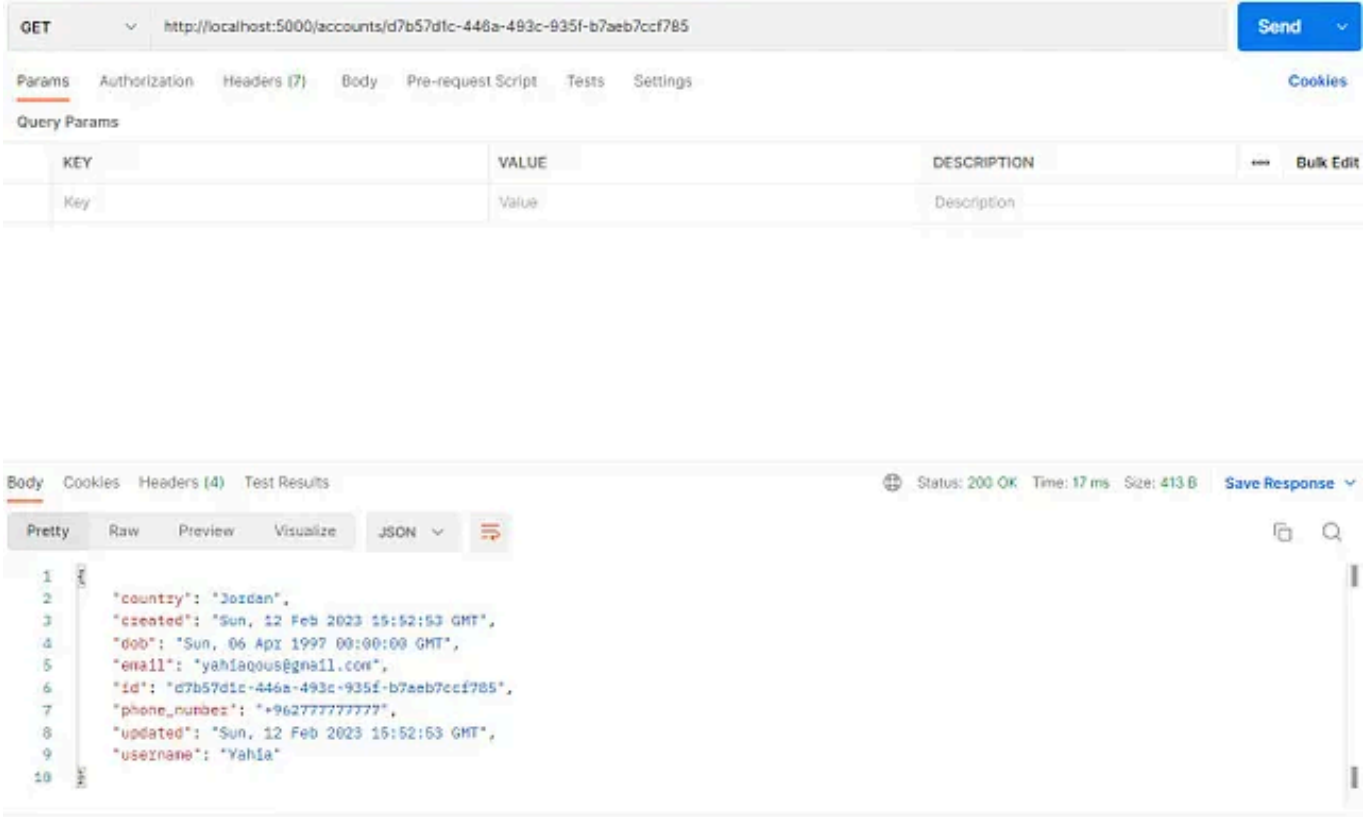
- Request Method: **GET**
- Request Link: **http://localhost:5000/accounts**



Postman - List All Accounts

3. **Get a Specific Account:**

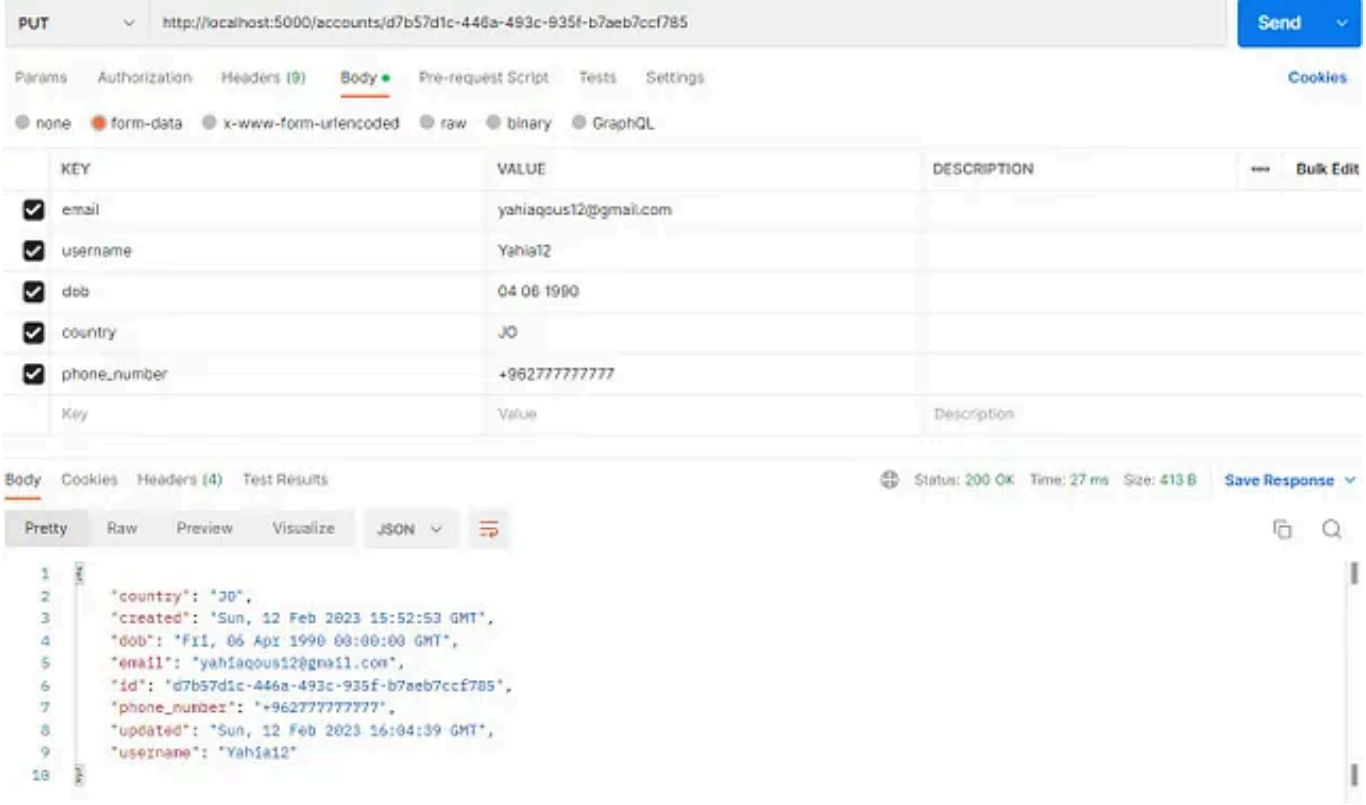
- Request Method: **GET**
- Request Link: **http://localhost:5000/accounts/ACCOUNT_ID**



Postman - Get a Specific Account

4. **Put a Specific Account:**

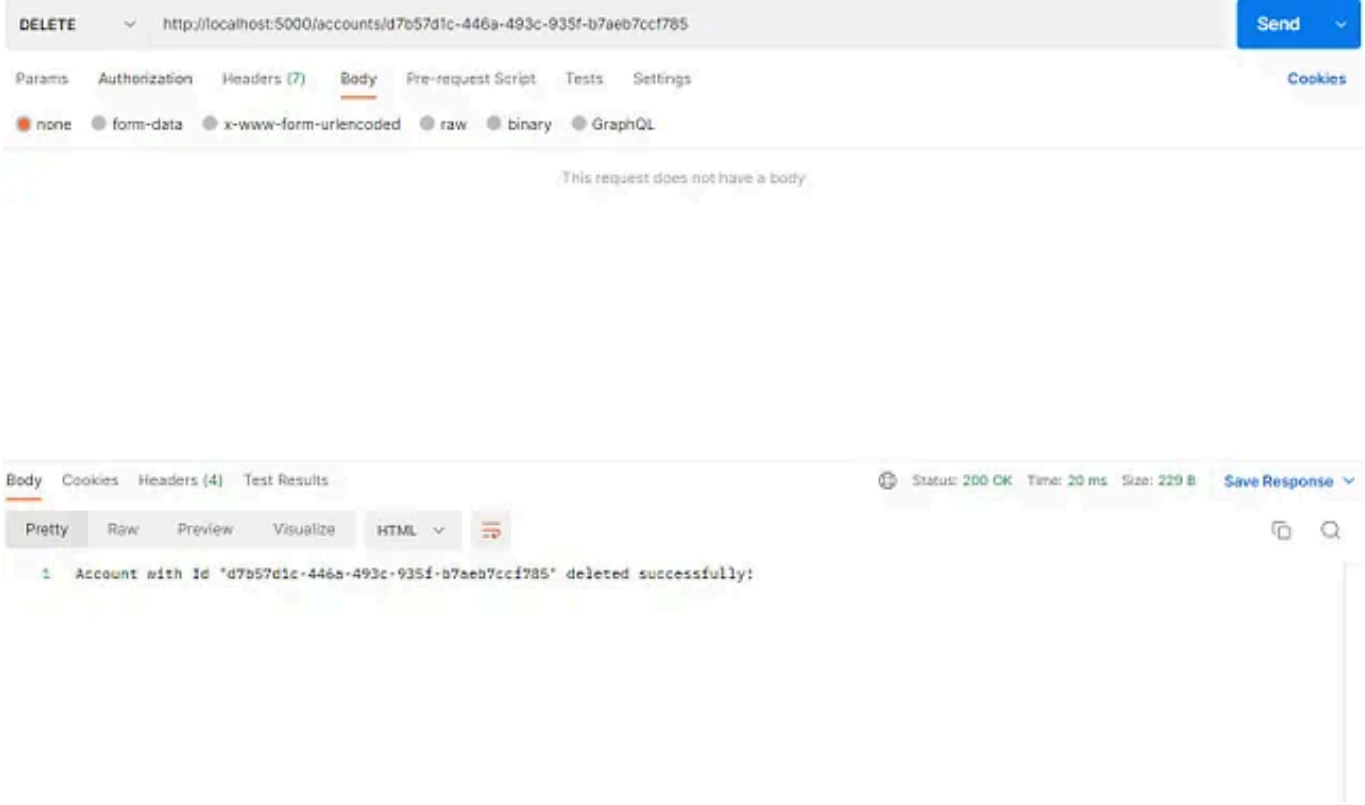
- Request Method: **PUT**
- Request Link: **http://localhost:5000/accounts/ACCOUNT_ID**
- Body Data in form-data:
  - email
  - username
  - dob
  - country
  - phone_number



Postman - Put a Specific Account

5. **Delete a Specific Account:**

- Request Method: **DELETE**
- Request Link: **http://localhost:5000/accounts/ACCOUNT_ID**



Postman - Delete a Specific Account

Get Started with SQLAlchemy Basic Relationships

Let's say we have multiple applications like **Accounts** & **Items** and we need to establish a relationship between their models!

> 📝 *Note: This is a short summary of the model's relationships, we'll go deeper into their CRUD operations in another article!*

1. One to Many Relationship

The Account may own many Items, but the Item is owned by one Account!

> 💡 *Tip: Use `ForeignKey` in the many side!*

```
class Account(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
      .
      .
      .
```

```
  # Relations:
    items = db.relationship("Item", back_populates='account')
```

```
class Item(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
      .
      .
      .

  # Relations:
  account_id = db.Column(db.String(100), db.ForeignKey("account.id"))
  account    = db.relationship("Account", back_populates="items")
```

2. Many to One Relationship

The Item may be owned by many Accounts, but the Account has only one Item!

> 💡 *Tip: Use `ForeignKey` in the many side!*

```
class Account(db.Model):
 id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
    .
    .
    .

# Relations:
 item     = db.relationship("Item", back_populates="accounts")
 item_id  = db.Column(db.String(100), db.ForeignKey("item.id"))
```

```
class Item(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False,
      .
      .
      .
```

```
  # Relations:
    accounts = db.relationship("Account", back_populates='item')
```

3. **One to One Relationship**

The Account can own one Item, and the Item owned by one Account!

> 💡 *Tip: Use `uselist=False` in one side & `ForeignKey` in the other side!*

```
class Account(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
     .
     .
     .

  # Relations:
    item = db.relationship("Item", back_populates='account', uselist=False)
```

```
class Item(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
     .
     .
     .

  # Relations:
    account    = db.relationship("Account", back_populates='item')
    account_id = db.Column(db.String(100), db.ForeignKey("account.id"), unique=Tru
```

4. **Many to Many Relationship**

The Account may own many Items, and the Item may be owned by many Accounts!

> 💡 *Tip: Use `Association` class with multi `ForeignKey`!*

```
class Association(db.Model):
  item         = db.relationship("Item", back_populates="accounts")
  account      = db.relationship("Account", back_populates="items")
  item_id      = db.Column('item_id', db.String, db.ForeignKey('item.id'), prima
  account_id   = db.Column('account_id', db.String, db.ForeignKey('account.id'),

  def toDict(self):
     return { c.key: getattr(self, c.key) for c in inspect(self).mapper.column_a

class Account(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
     .
     .
     .

# Relations:
  items = db.relationship("Association", back_populates='account')
```

```
class Item(db.Model):
  id = db.Column(db.String(50), primary_key=True, nullable=False, unique=True)
     .
     .
     .

# Relations:
  accounts = db.relationship("Association", back_populates="item")
Check out the Concept of backref and back_populate in SQLalchemy from this Stack
```